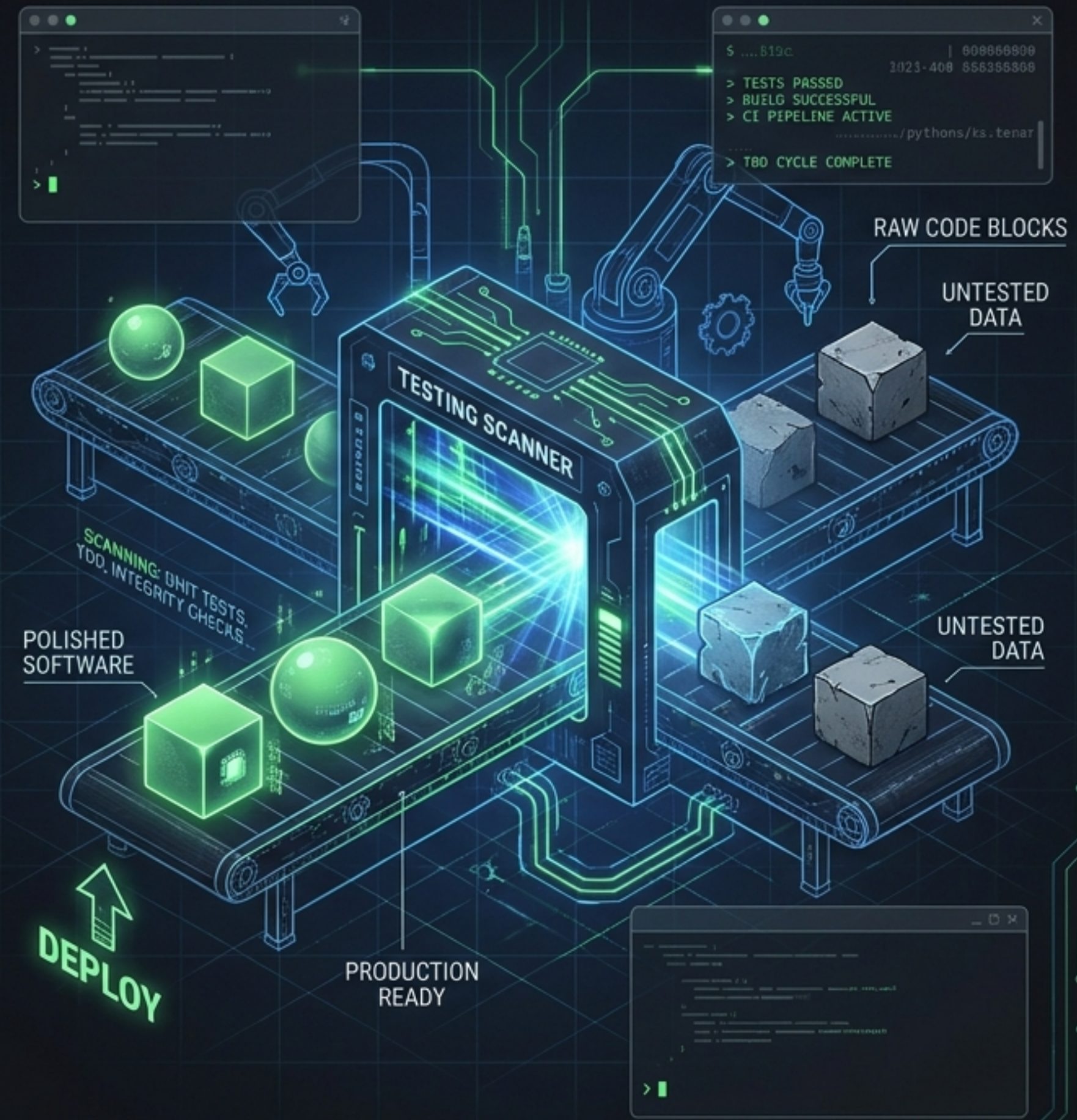


Testing in Python 3

From Fundamentals to Production Engineering

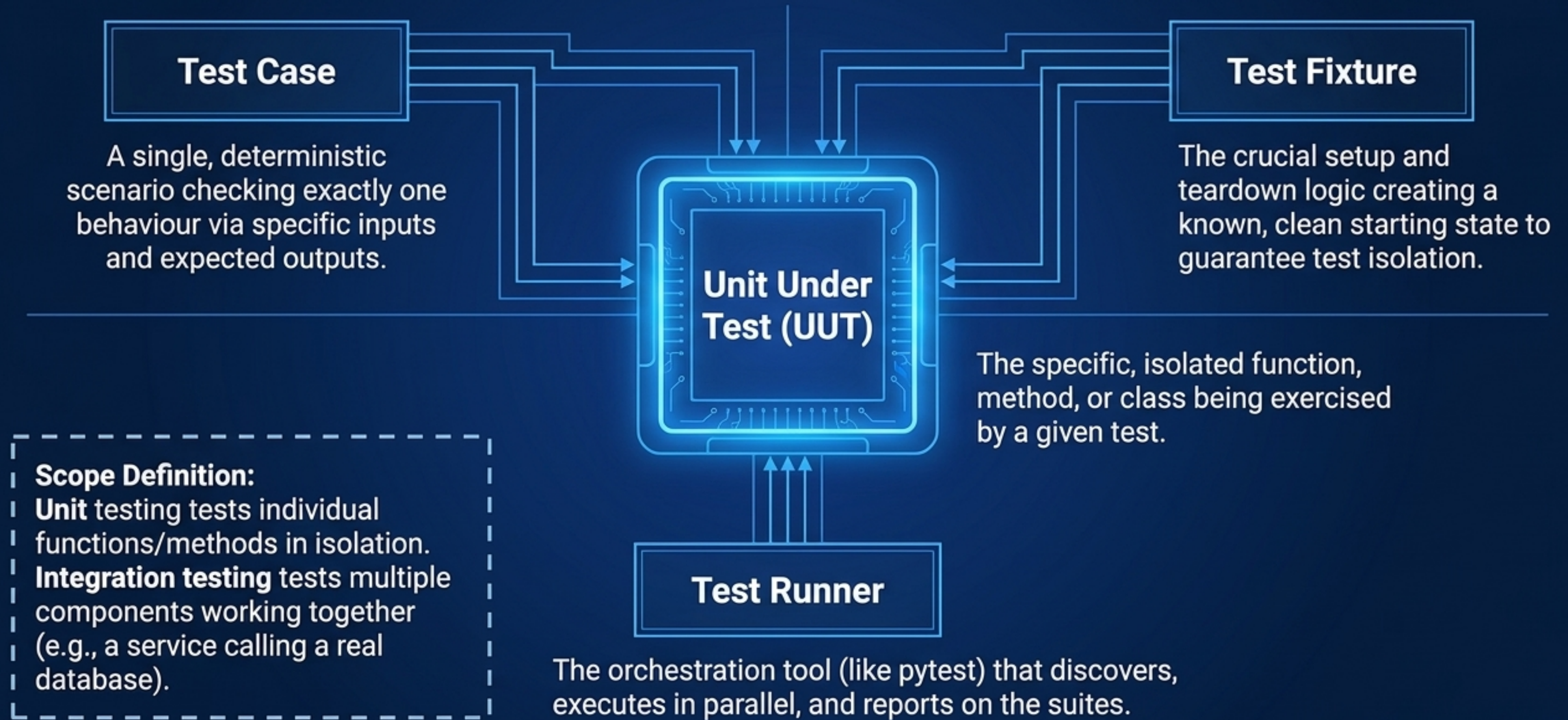
A masterclass in **unit testing**, **TDD**, **test doubles**, and **CI pipelines** for modern Python systems.



1. Why Automated Testing? (The Case for Confidence)

Without Tests (Fear & Friction)	With Tests (Speed & Confidence)
Changes are made cautiously, for fear of breaking something.	Changes are made confidently — the test suite immediately reports breakages.
Refactoring is risky and avoided.	Refactoring is safe — tests verify behaviour is preserved.
Collaboration is slow — reviewers cannot verify correctness.	Collaboration is fast — CI runs tests on every pull request.
Debugging is expensive — cause is often distant from symptom.	Debugging is cheap — failing test directly pinpoints the broken unit.
The Cost of Distance: Undetected bugs grow exponentially more expensive.	Executable Specifications: Tests codify precisely what code is supposed to do.

2. Unit Testing Fundamentals & Vocabulary



3. The AAA Pattern (Structuring Your Tests)

ARRANGE (Setup)

```
model = load_model('v1')  
data = [5.1, 3.5, 1.4]
```

Establish all preconditions—
instantiate objects, load test data,
and configure mock dependencies.

ACT (Execute)

```
prediction = model.predict(data)
```

Invoke the single unit under test,
capturing its return values or state
changes.

ASSERT (Verify)

```
assert prediction == 'setosa'  
>
```

Confirm the outcome against
expectations—checking values,
state changes, or raised exceptions.



Single Claim Principle: A good test makes a single falsifiable claim.
If assertions span multiple behaviours, split the test.

4. The unittest Framework (The Built-in Standard)

Class-Based Architecture:

Rigidly organised as methods within subclasses of `unittest.TestCase`.

State Management:

Relies on class-level methods to manage fixtures and resource isolation.

Rich Assertion Library:

Provides highly specific assertion methods (e.g., `assertAlmostEqual`).

```
test_legacy.py x
import unittest

class ModelTests(unittest.TestCase):
    def setUp(self):
        self.model = load_model()

    def tearDown(self):
        self.model.clear_cache()

    def test_prediction_output(self):
        result = self.model.predict(0)
        self.assertEqual(result, 1.0)
```

The Standard Library Anchor. Built directly into Python.

While largely superseded by modern frameworks, understanding its mechanics is required for enterprise codebases.

5. The pytest Framework (The Modern Standard)

unittest (Legacy)

Heavy class inheritance required.

Complex `.assertEqual()` syntax.

Generic failure messages.

Rigid class-level `setUp` and `tearDown`.

pytest (Modern)

Boilerplate Elimination: Plain Python functions radically increase readability.

Native Assertions: Uses Python's simple, built-in `assert` statement.

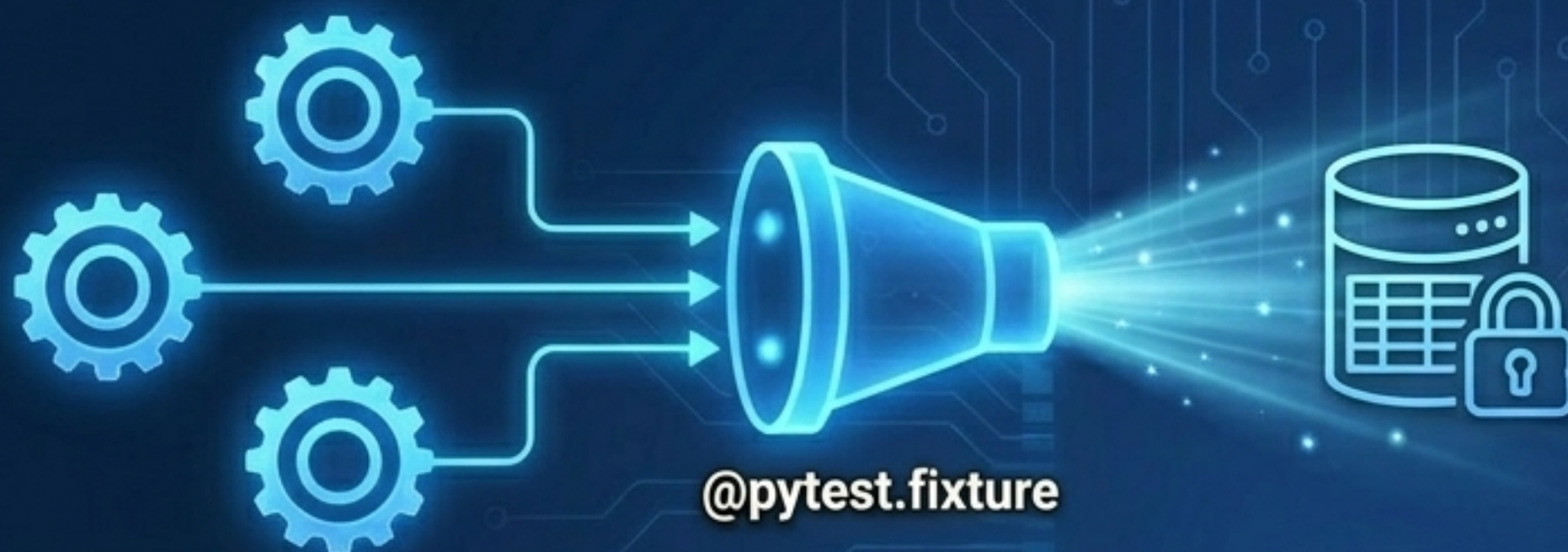
Advanced Diagnostics: Automatically generates rich diff outputs on failure, showing actual vs expected.

Composable Setup: Flexible, function-level dependency injection system.



Massive Ecosystem: Supported by over 500 plugins, seamlessly integrating with coverage tools, async code, and massive data science pipelines.

5. pytest Advanced: Fixtures & Parameterisation



The Power of Fixtures (Dependency Injection)

Test functions simply request fixtures by parameter name, allowing pytest to automatically wire the exact state required. Cleans up automatically.

Parameterisation Power

[[], ...]	...
[None, nan]	...
[1000, -50]	...
[{"key": "val"}, ...]	...

Diverse Data Inputs

Eliminates repetitive boilerplate.
The ideal tool for verifying data preprocessing pipelines across edge by running a single function through a matrix of inputs and expected outputs.

@pytest.mark.parametrize



Test Result 1

Test Result 2

Test Result 3

⋮

Test Result N

6. Test Driven Development (TDD)

● **REFACTOR (Improve)**

Safely restructure the code and improve logic, knowing the green tests will catch any regression.



● **RED (Fail)**

Write a failing test to confirm exactly what new behaviour is required.

● **GREEN (Pass)**

Write only the absolute minimum implementation required to make the test pass.

Design First, Code Second.

- TDD forces developers to perfectly define the public interface before writing implementation.
- **Built-In Regression Suite:** The natural byproduct is a comprehensive safety net guarding against future breakages.

“TDD is a way of managing fear during programming.” – Kent Beck

7. Continuous Integration (Automated Pipelines)



The Automated Quality Gate

Automatically triggers tests upon every Git push, verifying integration in minutes.

Stopping Silent Failures

Prevents broken code or statistical shifts in ML pipelines from entering production.

Routine Integration

Transforms integration from a multi-day crisis into a routine, automated operation.

8. Test Doubles Overview (Isolating Dependencies)



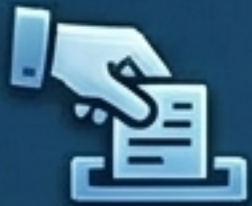
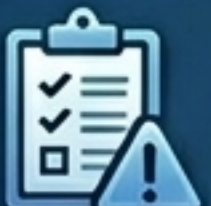
 **The Stunt Doubles of Code:** Objects constructed purely to stand in for real dependencies during a test execution.

 **Bypassing the Slow:** Swaps out network services or heavy ML training loops for instant alternatives.

 **Isolating the Unavailable:** Replaces hardware sensors or production databases inaccessible in CI.

 **Controlling the Unpredictable:** Tames randomness, time, or live feeds by returning fixed values.

8. The Test Doubles Matrix

	DUMMY	A simple placeholder object passed strictly to satisfy a parameter signature, never actually used by the code.
	STUB	Provides predefined, hard-coded answers to calls made during the test, feeding controlled inputs into the unit.
	FAKE	Contains actual working logic but takes vast shortcuts (e.g., an in-memory SQLite database instead of a live Postgres cluster).
	SPY	Wraps a real object to silently record its interactions (method calls, argument counts) for subsequent assertion.
	MOCK	Specifies exactly what interactions are expected upfront, automatically failing the test if unexpected calls occur.

**Core Rule: Use Stubs to provide controlled INPUTS to the unit.
Use Mocks to verify output BEHAVIOUR and interactions.**

9. Code Coverage & Branch Coverage

STATEMENT COVERAGE



BRANCH COVERAGE



The 100% Trap

Chasing perfect coverage often wastes engineering time on testing trivial getters, setters, and structural boilerplate rather than verifying meaningful behaviour.

Production Target: For data science and ML projects, target 80-85% statement coverage with branch coverage enabled. Focus coverage effort on business logic, data transformation functions, and error handling paths. Do not target 100% — it incentivises writing tests for trivial getters/setters and `_repr_` methods rather than for meaningful behaviour.

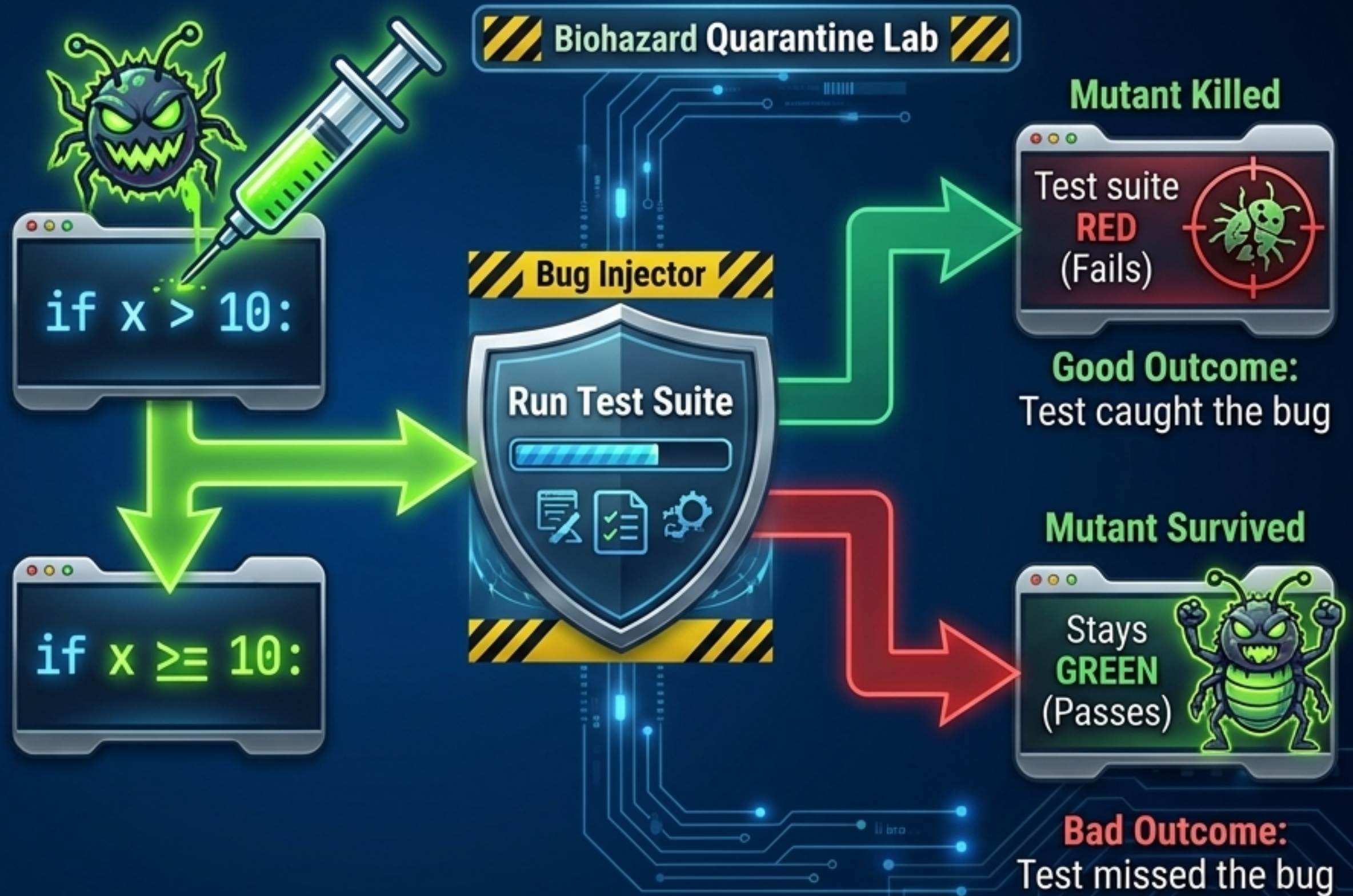
9. Mutation Testing (Testing Your Tests)

Testing Your Tests:

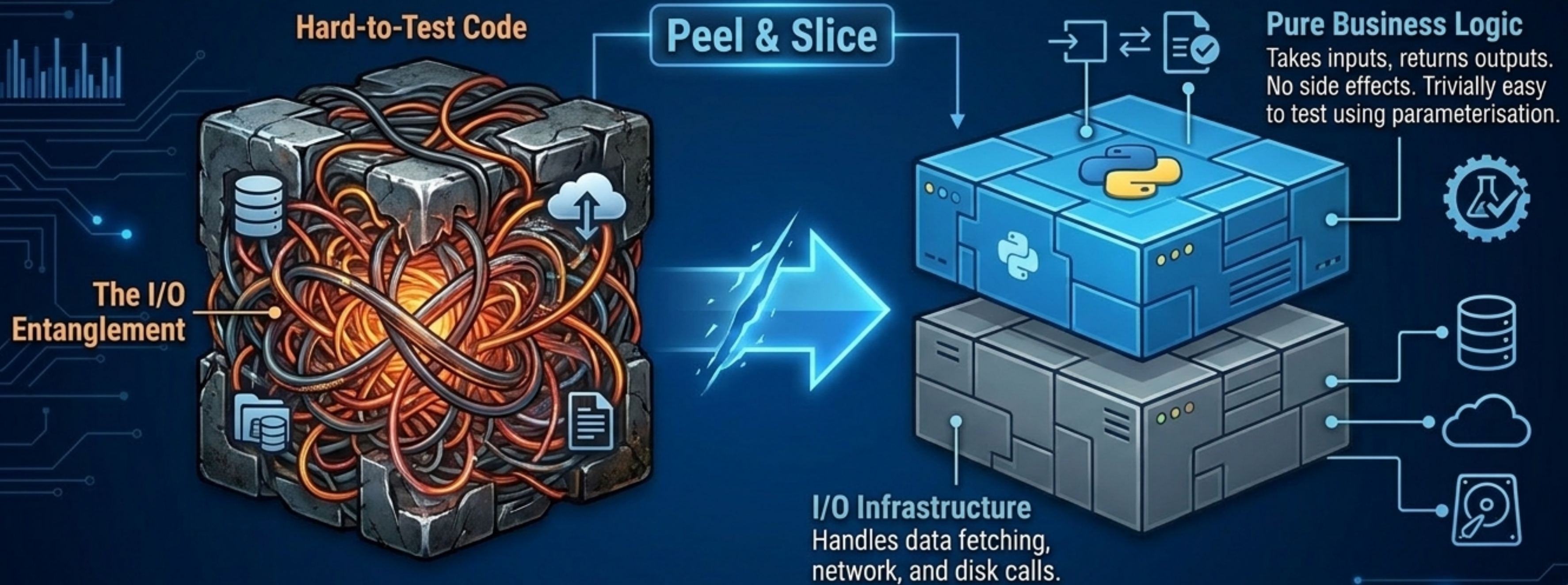
Measures test quality by deliberately injecting small, syntactic faults into production code.

Coverage Without Comprehension: Exposes tests that execute code but fail to actually verify its behaviour.

The Gold Standard: Achieving an 80%+ mutation kill rate proves the suite operates as a highly sensitive fault-detection mechanism.



10. Refactoring Hard-to-Test Code



Caution: Monkey patching dependencies at runtime via `unittest.mock.patch` is a brittle last resort. Over-reliance signals a need for dependency injection.

11. Summary (The Testing Reference Guide)

When testing a...	Use this approach...
Pure function (no I/O, no side effects)	Parameterised pytest tests with multiple input/output pairs
Function that calls an external API	Mock the HTTP client, or use VCR.py for self-initialising fakes
Function that writes to a database	Use an in-memory Fake database or a real test DB with fixture teardown
Collaborator's interaction correctness	Utilise a strict Mock with <code>assert_called_once_with()</code>
Complex rendered outputs (JSON/HTML)	Deploy Approval Testing to compare against pre-verified golden snapshots

Final Rule: Every line of production code should exist because a test required it.